CS x290/ECE x100
Conte

Instruction-Level Parallelism (ILP)

Copyright © 1995-2021 T. M. Conte

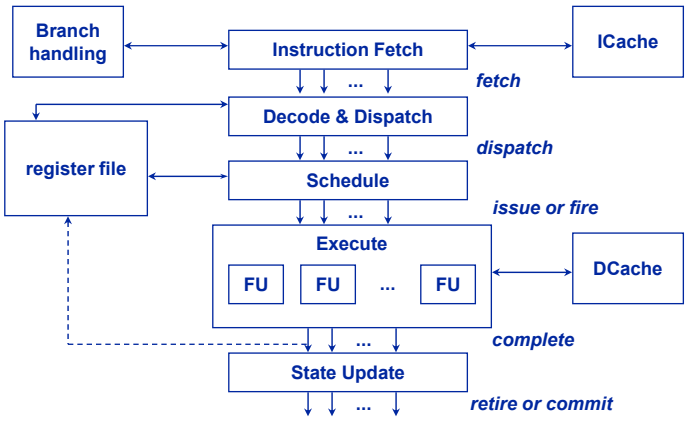
1

Limits of Pipelining

- CPU_time = IC x CPI x CT
- Scalar pipelining
  - CPI ≥ 1 (Flynn's bottleneck)
  - ∴ reduce CT further, increase instructions per *second*
- Problem: limits of (single instruction per cycle) pipelining
  1. Continue to reduce CT, but can only break logic up so far
  2. Latch overhead and clock skew start to dominate
     - Overhead becomes a larger component of cycle
     - Pipelining eventually becomes inefficient
  3. Deep pipelines are ineffective when there are hazards

2

View of superscalar processor



ILP-5

5

Roadmap for this Chapter

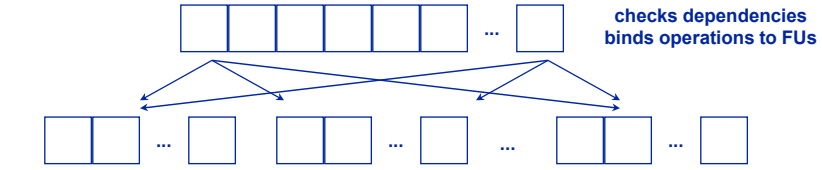
- Dispatch and Schedule hardware algorithms
- State update designs
- Instruction fetch designs
- Doing it all in a completely different way…(more details to follow)

ILP-6

6

Dispatch and scheduling

- Dispatch and schedule units maintain queueues:



entries for:
function unit #1 *function unit #2* *function unit #N*

ILP-7

7

A little about function(al) units

- Function(al) units = FUs
- Typical latencies and why:

Type	Latency	Why
Integer ADD	1	The ALU is critical
FP MUL	3	Stage 1: Denormalize; Stage 2: add exp, multi mantissas; Stage 3: Normalize
FP ADD	2	Stage 1:Denormalize; Sage 2: Add and normalize (1 cycle)
Load (LW)	2	The cache pipeline
Store (SW)	1	Store buffering

ILP-8

8

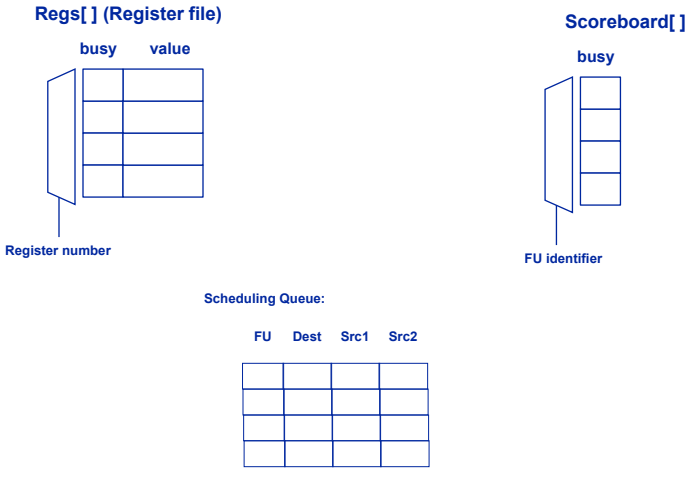
A classification of approaches

- Fire (issue) (F) either in-order (I) or out-of-order (O)
- Complete either in-order (I) or out-of-order (O)
- So FICI are pipelines like the 5-stage pipeline from last section
- The Cray-1 (simple interlocking) was FICO
- The IBM 360/91 was FOCO
- (Note "FOCI" isn't a useful configuration!)

ILP-10

10

A dynamic scheduling algorithm: Simple interlocking (FICO) Hardware view--



ILP-13

13

A dynamic scheduling algorithm: Simple interlocking (FICO)

```
1. [Dispatch/Scheduling unit (combined unit)]
      for each Inst in SchedQueue do:
            if ( !Regs[Inst.Src1].Busy
                  AND !Regs[Inst.Src2].Busy
                  AND !Regs[Inst.Dest].Busy
                  AND !Scoreboard[Inst.FU].Busy )
            then
                  Scoreboard[Inst.FU].Busy = True
                  Regs[Inst.Dest].Busy = True
                  // Issue instruction to function unit Inst.FU
            else
                  exit loop          // halt issuing

2. [Execution unit, at completion of instruction Inst]
      (a) Regs[Inst.Dest].Busy = False
      (b) if (Inst.FU pipeline advances and now first stage is free) then
            Scoreboard[Inst.FU].Busy = False
```


14

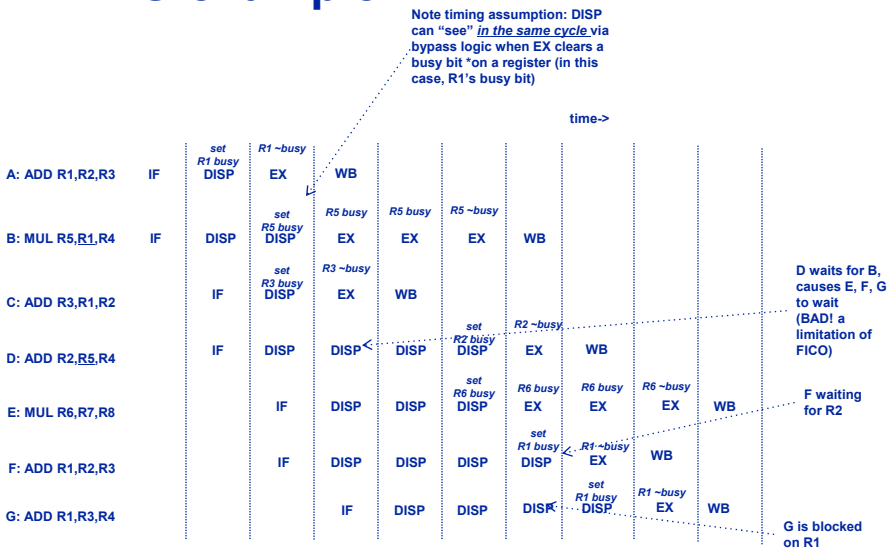
A dynamic scheduling algorithm: Simple interlocking (FICO) (simplified)

```
1. [Dispatch/Scheduling unit (combined unit)]
      Scan SchedQueue in FIFO order
      For each Inst in SchedQueue:
            if ( Both source regs of Inst are not busy
                  AND destination reg is not busy
                  AND scoreboard says FU is not busy )
            then
                  Mark FU busy in Scoreboard
                  Dest Inst's dest reg as busy
                  // Issue Inst
            else
                  exit loop          // halt issuing

2. [Execution unit, at completion of instruction Inst]
      (a) Clear busy bit of dest reg of Inst
      (b) if (Inst.FU pipeline advances and now first stage is free) then
            clear the scoreboard bit for Inst.FU
```


15

FICO example



16

How FICO enforces dependencies

- **Flow dependence**

Scheduling/dispatch queue

R1.Busy set = 1

A: ADD R1, R2,R3

B: ADD R4, R1, R3

R1.Busy checked: halt firing

scanned in program order

- **Anti-dependence**

Scheduling/dispatch queue

R1's new value will be written after B's execution begins

A: ADD R4, R1, R3

B: ADD R1, R2, R3

R1's old value read during scheduling/dispatch

- **Output dependence**

Scheduling/dispatch queue

A: MUL R1, R2, R3

B: ADD R1, R4, R5

R1.Busy set = 1

R1.Busy checked: halt firing

17

Tomasulo's Algorithm

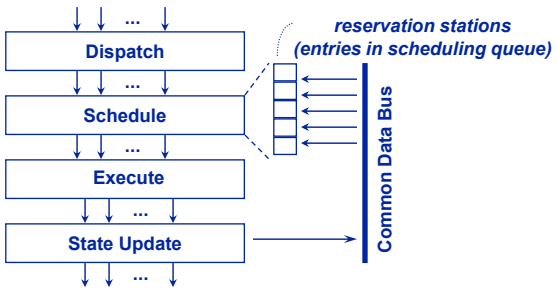
- Robert Tomasulo consulted on the IBM 360 model 91
  - Born of necessity: model 85 vs. 91 competition (the '91 lost…)
- Used in IBM 360/91 floating-point unit
  - Many long-latency operations
  - Need dynamic scheduling: mitigate long stalls
- ISA specified only 4 floating-point registers
  - Need register renaming: with only 4 registers, false dependencies pop up quickly
  - Renaming was a requirement!

ILP-20

20

Reservation stations and the Common Data Bus

- Scheduling queue entries become "reservation stations": places to park instructions waiting on specific values to be calculated
- Announcements of specific values are broadcast on the "Common Data Bus"
  - In original IBM 360 model 91, there was one CDB. Today there are often several (renamed a less awkward term: "result buses")



ILP-21

21

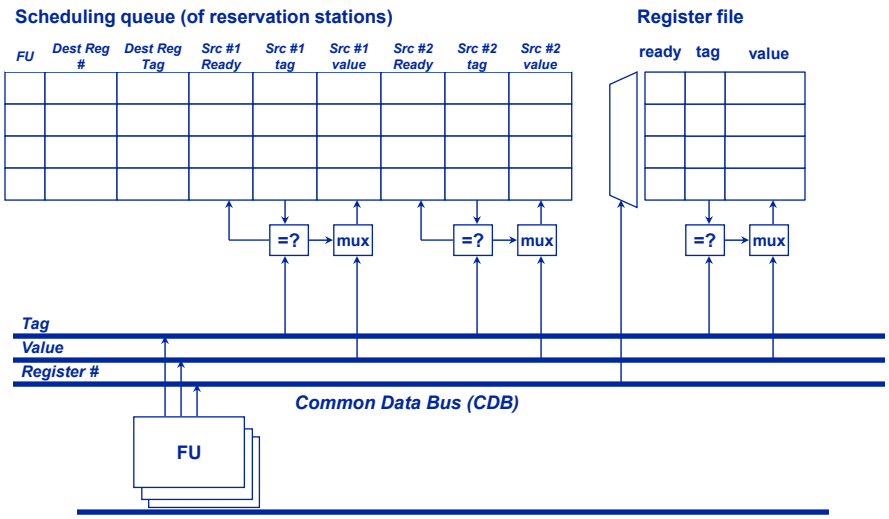
Renaming registers

- Problem: anti, output dependencies can cause many cycle stalls (recall IPC ~ # regs)
- Solution: Take advantage of reservation stations and common data bus to tag values with new "names" different from register #'s
  - Assign "tags" (new names) to destination register names in program order (during Dispatch)
  - Add "tag" fields to reservation stations, common data bus, register file
- Note well: renaming registers and out-of-order issue are separate capabilities, although they are often combined

ILP-22

22

Tomasulo Hardware View:



ILP-23

23

High level Tomasulo agorithm

- Dispatch: For each inst, put in sched queue: looks up src regs for value or tag, assigns a tag to dest reg and marks it not ready
- Scheduling: manages the sched queue: listens to bus to update ready bits/values, fires insts in sched queue that have both source regs ready and FU free
- Complete: broadcasts result on bus, frees slot in sched queue
- Reg file: if bus tag matches reg entry, updates value and marks reg ready

24

Tomasulo algorithm, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Insttruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.Dest = I.Dest
        (e) for i = 0..1 do
                if (Regs[I.src[i]].Ready = True) then
                    RS.Src[i].Value = Regs[I.src[i]].Value
                    RS.Src[i].Ready = True
                else
                    RS.Src[i].Tag = Regs[I.src[i]].Tag  // Copy the tags from the RF
                    RS.Src[i].Ready = False
        (f) Regs[I.Dest].Tag = unique tag id    // tag (rename) the destination
        (g) RS.DestRegTag = Regs[I.Dest].Tag
        (h) Regs[I.Dest].Ready = False                        // value ready only after execution

    else exit loop // stop dispatching if scheduling queue is full
```


25

Tomasulo algorithm, part 2

```
[ Scheduling unit ]
for each RS = entry in SchedQueue do:
      (a) for i = 0..1 do:
            if (CDB.Tag = RS.Src[i].Tag) then          // broadcast on CDB matches stored tag
                  RS.Src[i].Ready = True
                  RS.Src[i].Value = CDB.Value
      (b) if (RS.Src[0].Ready = True AND RS.Src[1].Ready = True ) // WAKEUP: If all registers are
ready...
            and Scoreboard[RS.FU].Busy = False) then
                  Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
                  Issue the instruction at RS on FU
```

step (b) is called "wake-up"
when more than one instruction is ready, picking between them is called "select" (here we're using FIFO as a select algorithm)

ILP-26

26

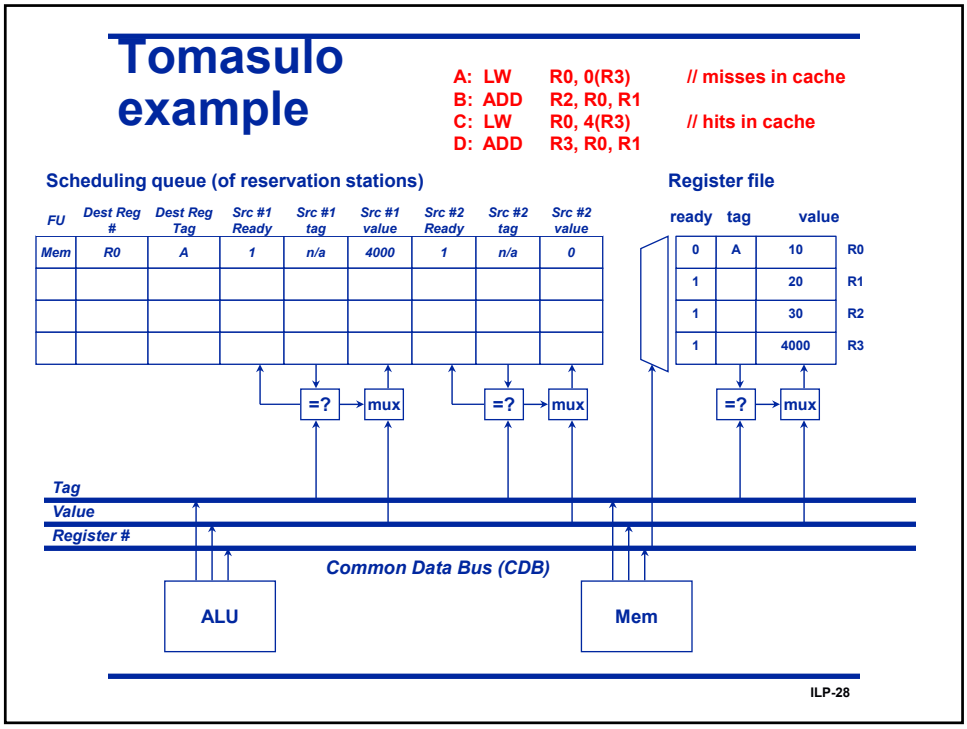
Tomasulo Algorithm, part 3

```
[ Execution unit, for function unit FU]
      if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
      if (CDB.Busy = False) then                   // Broadcast the instruction results
            (a) CDB.Busy = True
            (b) CDB.Tag = I.Tag
            (c) CDB.Value = I.Value
            (d) CDB.Reg = I.Dest
            (e) if (I.FU is not pipelined) then
                  Scoreboard[I.FU].Busy = False
            (f) Delete I from Sched Queue

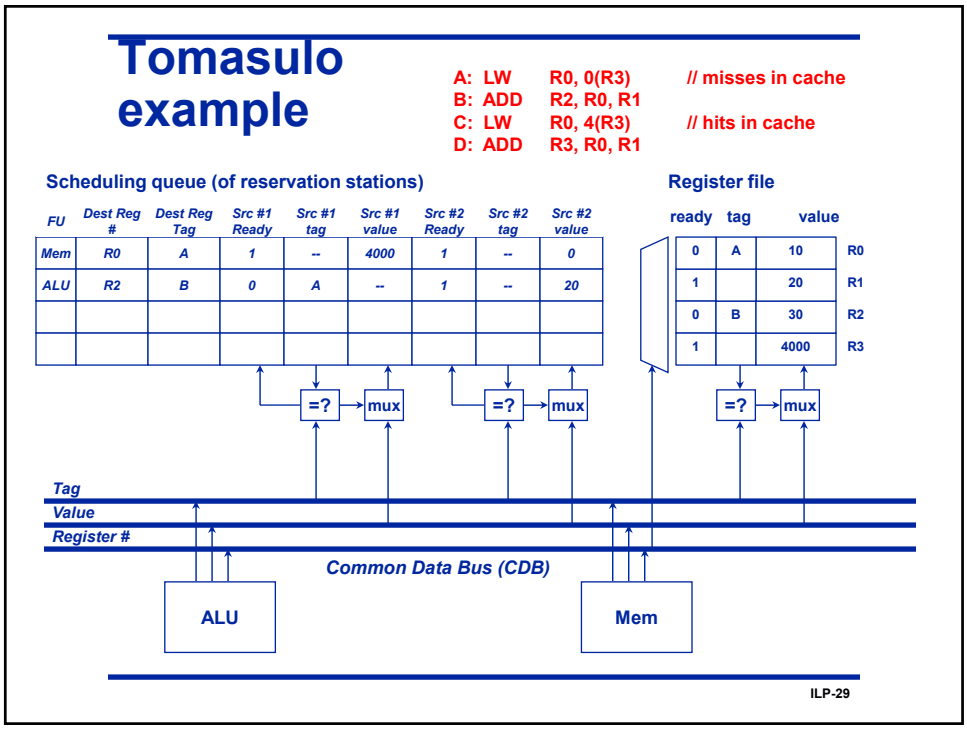
[ Register file ]
      if (CDB.Busy = True and
      CDB.Tag = Regs[CDB.Reg].Tag) then
            (a) Regs[CDB.Reg].Ready = True         // Update the RF contents
            (b) Regs[CDB.Reg].Value = CDB.Value
            (c) CDB.Busy = False
```

ILP-27

27


Tomasulo example
A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1
Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
Mem R0 A 1 n/a 4000 1 n/a 0
ready tag value
0 A 10 R0
1 20 R1
1 30 R2
1 4000 R3
=? mux
Tag
Value
Register #
Common Data Bus (CDB)
ALU
Mem
ILP-28

28


Tomasulo example
A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1
Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
Mem R0 A 1 -- 4000 1 -- 0
ALU R2 B 0 A -- 1 -- 20
ready tag value
0 A 10 R0
1 20 R1
0 B 30 R2
1 4000 R3
=? mux
Tag
Value
Register #
Common Data Bus (CDB)
ALU
Mem
ILP-29

29

Tomasulo example

A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1

Scheduling queue (of reservation stations)

FU	Dest Reg #	Dest Reg Tag	Src #1 Ready	Src #1 tag	Src #1 value	Src #2 Ready	Src #2 tag	Src #2 value
Mem	R0	A	1	--	4000	1	--	0
ALU	R2	B	0	A	--	1	--	20
Mem	R0	C	1	--	4000	1	--	4

Register file

	ready	tag	value
R0	0	C	10
R1	1		20
R2	0	B	30
R3	1		4000

=? mux =? mux =? mux

Tag
Value
Register #

Common Data Bus (CDB)

ALU Mem

ILP-30

30

Tomasulo example

A: LW R0, 0(R3) // misses in cache
B: ADD R2, R0, R1
C: LW R0, 4(R3) // hits in cache
D: ADD R3, R0, R1

Scheduling queue (of reservation stations)

FU	Dest Reg #	Dest Reg Tag	Src #1 Ready	Src #1 tag	Src #1 value	Src #2 Ready	Src #2 tag	Src #2 value
Mem	R0	A	1	--	4000	1	--	0
ALU	R2	B	0	A	--	1	--	20
Mem	R0	C	1	--	4000	1	--	4
ALU	R3	D	0	C	--	1	--	20

Register file

	ready	tag	value
R0	0	C	10
R1	1		20
R2	0	B	30
R3	0	D	4000

=? mux =? mux =? mux

Tag
Value
Register #

Common Data Bus (CDB)

ALU Mem

ILP-31

31

Tomasulo problems / solutions

- CDB limits # instruction that can complete per cycle to one instruction
  - Solution: Have N CDBs. Called "result buses" because they are no longer "common" data buses. Now can complete N instructions per cycle
- The scheduling queue needs a fully associative lookup for each result bus
  - Very expensive!
  - Solution…

ILP-32

32

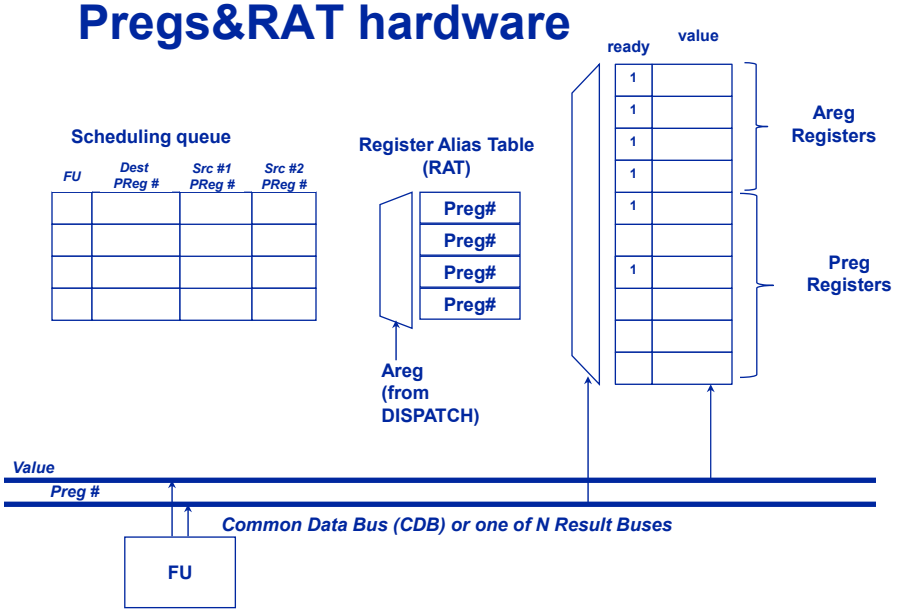
Alternative Implementation: Physical regs and RAT

- Instead of TAGs, use real registers
  - Extend the register file with and PHYSICAL (UNARCHITECTED) registers (Pregs) in addition to traditional (architected) registers, called "Aregs"
  - (Architected– ie, traditional– registers are called "Aregs")
  - Use a TABLE to map most recent Preg to Aregs – called the Register Alias Table (RAT)

ILP-33

33

Pregs&RAT hardware



ILP-34

34

PRegs&RAT

- At DISPATCH of instruction "*I*"
  - Say D = the Areg dest register of *I*: pick an unused Preg index *"P"*
  - Set RAT[D] = P
- At scheduling unit for entry *RS*:
  - Wake up *RS* if register for each source of *RS* is ready
  - Scheduler (or FU) reads the register file for value
- On instruction completion of *I*:
  - Write the result into Preg in the physical register file part and mark it as "Ready"
- The Areg portion of the register file is used when freeing Pregs (details to follow…)

ILP-35

35

Tomasulo with Pregs&RAT, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
      if (SchedQ is not full AND there are free Pregs) then
            (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
            (b) Delete I from DispQ                               // Instruction I is now "dispatched"
            (c) RS.FU = I.FU
            (d) for i = 0..1 do
                  RS.SRC[i] = RAT[I.srci]
            (e) RAT[I.Dest] = Find a free Preg #   // rename the dest
            (f) RS.Dest_Preg_index = RAT[I.Dest]
            (g) Regs[ RAT[I.Dest] ].Ready = False
      else exit loop // stop dispatching if scheduling queue is full or there are no free PRegs


[ Scheduling unit ]
for each RS = entry in SchedQueue do:
      if (Regs[ RS.SRC[0] ].Ready = True AND
         Regs[ RS.SRC[1] ].Ready = True ) // WAKEUP: If all registers are ready...
            and Scoreboard[RS.FU].Busy = False) then
                  Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
                  Issue the instruction at RS on FU
```


36

Tomasulo with Pregs&RAT part 2

```
[ Execution unit, for function unit FU]
      if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
      if (CDB.Busy = False) then                          // Broadcast the instruction results
            (a) CDB.Busy = True
            (b) CDB.Value = I.Value
            (c) CDB.PReg = I.Dest_Preg
            (d) if (I.FU is not pipelined) then
                  Scoreboard[I.FU].Busy = False
            (e) Delete I from Sched Queue

[ Register file ]
      if (CDB.Busy = True) then
            (a) Regs[CDB.PReg].Ready = True           // Update the RF contents
            (b) Regs[CDB.PReg].Value = CDB.Value      // update the Preg
            (c) CDB.Busy = False
```


37

Pregs/RAT vs. Tomasulo

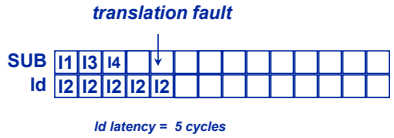
- Pregs/RAT has no associative lookup in the scheduling queue (or anywhere else), no "broadcast" to listen to
- But Tomasulo doesn't need to check the register file for all source registers, for each entry, on every cycle
- Problem: when do you free Pregs??
  - Answer in a little bit…

ILP-38

38

State Update

I1 SUB R1, R4, R5
I2 LW R2, R3(R4)
I3 SUB R3, R1, R5
I4 SUB R1, R3, R1



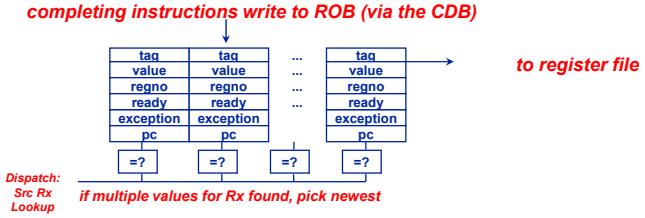
where do you restart the code after the page fault is serviced?
I3 already destroyed R3, I4 destroyed R1 so I3 cannot be re-executed

- Problem: No consistent state
  - OS needs "resume PC" to restart process after fault is handled
  - Also called: *Imprecise Interrupts*
- Definitions:
  - Fault: An excepting instruction that should be retried after exception is fixed
  - Trap: An excepting instruction that should *not* be retried after exception
  - Exception handler: routine that fixes exceptions
  - An architectural state: (1) register contents, (2) memory contents, (3) PC value

ILP-40

40

Solution #1: Reorder Buffer with Bypass (ROB w/bypass)



- Add comparators to search reorder buffer for src regs
  - The search uses source register numbers of the dispatching instruction: replaces register file lookup in basic Tomasulo
  - Search from newest to oldest ROB entry
  - If no match, get the value from the register file
  - See next slide for algorithm modifications
- Advantage: No longer have to wait until retirement for destination register to be updated
- Register file only stores the values, no additional meta data
- Problem:
  - Comparators are expensive
  - ROB now similar to fully-associative cache: a cycle time stretcher

Tomasulo & ROB w/bypass

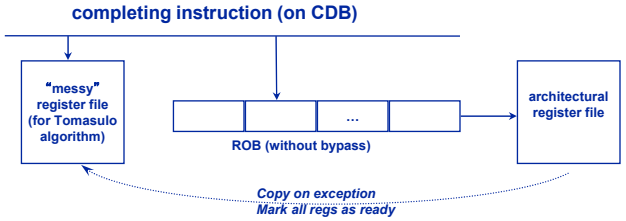
```
[Dispatch unit ] for all I in DispatchQueue do:
    if (SchedQueue is not full) then
        (a) Add I to first free slot of SchedQueue  (="RS")        // Call this RS below
        (b) Delete I from DispatchQueue                                  // Inst is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.DestReg = I.Dest
        (e) for i = 0 to Max Source Reg-1
                for j = ROB.tail to ROB.head                       // search for source registers in ROB
                    if (ROB[j].regno = RS.Src[i]) then
                            found_in_rob = True
                            if (ROB[j].Ready = True) then
                                    RS.Src[i].Value = ROB[j].Value
                                    RS.Src[i].Ready = True
                            else
                                    RS.Src[i].Tag = ROB[j].Tag  // Copy the tags from the ROB
                                    RS.Src[i].Ready = False
                            break
                if (found_in_rob = True) then continue  // go to next source reg
                else // no write of reg in ROB, get the value from the register file (because instruction retired long ago!)
                        RS.Src[i].Value = Regs[RS.Src[i]].Value
                        RS.Src[i].Ready = True

        (f) ROB[tail].Tag = unique tag id          // tag (rename) the dest, could just use the ROB index ('tail') as tag#
        (g) ROB[tail].Regno = I.Dest
        (h) ROB[tail].Ready = False
        (i) ROB[tail].Exception = False
        (j) tail++
```

Tomasulo & ROB w/bypass (cont.)

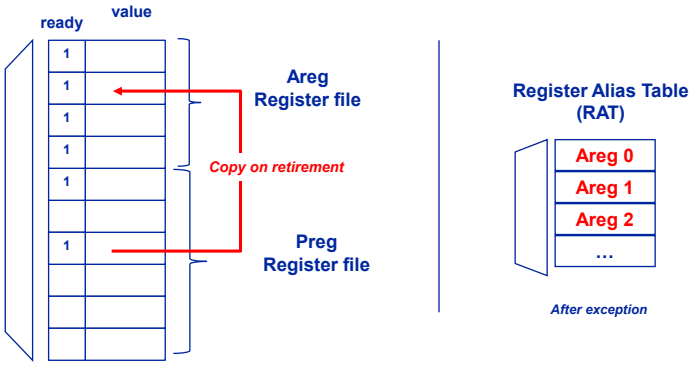
```
[At instruction completion of instruction I at index 'index' in the ROB]:
        (a) ROB[index].Value = I.Value
        (b) ROB[index].Ready= True
        (c) If exception for instruction I then
                ROB[index].Exception = true
[At instruction retire, at state update unit]:
        if (ROB[head].exception = false) then
                Regs[ROB[head].Regno].Value = ROB[head].Value
        else /* exception occurred */
                (a) resume_pc =  ROB[head].pc
                (b) tail = head // flush the ROB
                (b) call exception handler
                (c) if trap, then pc = resume_pc+4
                    else /* fault */ pc = resume_pc
```


Solution #2: ROB w/Future File



- Changes vs. ROB w/bypass are:
  - Keep the original Tomasulo register file (called "messy")
  - Completing instructions broadcast on CDB and update the messy RF as in original Tomasulo algorithm
  - Dispatch checks "messy" register file (also called *Future File* by Smith/Pleszkun in 1988) just like original Tomasulo algorithm.
  - Before exception handling, architectural register file is copied to messy register file
- The most commonly implemented scheme

ROB w/Future File: Pregs/RAT version



1. When instruction reaches head of ROB, copy Preg to corresponding Areg
2. On exception, change RAT to point to corresponding Aregs

46

Freeing PRegs via ROB

THIS IS HOW YOU FREE Pregs:

- At DISPATCH of instruction "*I*"
  - Allocate slot in ROB for *I* at tail of ROB
  - ROB[tail].Previous_preg = RAT[I.Dest_areg]

  (Rest of Dispatch is the same as prior PReg/RAT)
- On instruction retirement of *I* at head of ROB:
  - Mark ROB[head].Previous_preg as free

47

Pregs/RAT with Preg freeing

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full AND there are free Pregs available) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Insttruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) for i = 0..1 do
                RS.SRC[i].reg_index = RAT[I.srci]
        (e) ROB[tail].Dest_Areg = I.Dest
        (f) ROB[tail].Prev_Preg = RAT[I.Dest]
        (g) RAT[I.Dest] = Find a free Preg #   // rename the dest
        (h) RS.Dest_Preg = RAT[I.Dest]
        (i) Regs[ RAT[I.Dest] ].Ready = False
        (j) ROB[tail].Ready = False
        (k) ROB[tail].Exception = False
        (l) tail++


    else exit loop // stop dispatching if scheduling queue is full
```

ILP-48

48

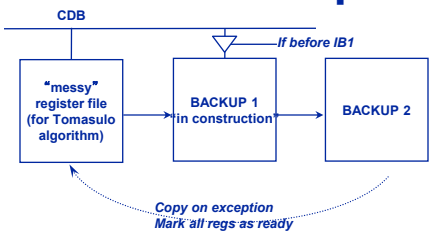
Pregs/RAT with Preg freeing

```
[At instruction retire, at state update unit]:
        if (ROB[head].exception = false) then
                (a) Regs[ROB[head].Dest_Areg].Value = ROB[head].Value
                (b) Mark ROB[head].Prev_Preg as Free in Preg free list
        else /* exception occurred */
                (a) resume_pc =  ROB[head].pc
                (b) tail = head // flush the ROB
                (b) call exception handler
                (c) for Areg = 0 to max_Areg do
                        RAT[Areg] = Areg
                (d) if trap, then pc = resume_pc+4
                    else /* fault */ pc = resume_pc
```

ILP-49

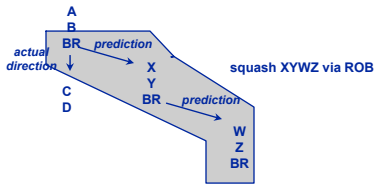
49

Solution #3: Checkpoint repair



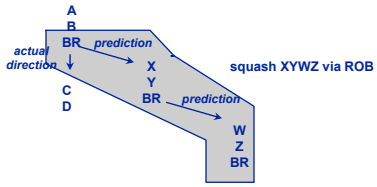
- Periodically mark an instruction barrier (IB1) at dispatch
- All instructions that complete that are before IB1 write to both messy and Backup 1
- Once all instructions before IB1 have completed, copy Backup 1 to Backup 2, Messy to Backup 1, set IB2 = IB1, and pick a new IB1
- Note that all loads and stores are buffered until backup 1 copied to backup 2
- If an exception occurs, copy Backup 2 to messy and resume at IB2 for Backup 2 after exception => may have to re-execute some instructions between IB and where the exception occurred
- Note: for a TRAP, set a new IB1 right before the TRAP and don't handle the trap until Backup 1 is complete
- Invented in 1988 by Patt and Hwu

Speculative execution



- Add branch prediction to fetch unit
- Could delay dispatch until a branch's behavior is known, OR...
- When branch is predicted, begin executing instructions speculatively until outcome is known
- Need a technique to recover from miss-speculation so speculative instructions do not hurt processor state...

Miss-speculation recovery



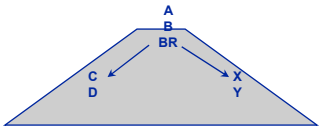
- Conceptually like "mini-exceptions"
- ROB recovery:
  - Flush all instructions from branch to tail of reorder buffer
  - No need to wait until branch is retired
  - Resume execution at correct path of branch
- Checkpoint repair recovery:
  - On branch prediction, start a new IB and copy Messy to Backup 1.
  - On mis-speculation recovery, do not write back results of any instructions after IB
  - Instead, just copy Backup 1 to Messy and resume executing instructions after the branch

ILP-52

52

Alternative to speculation: Multipath execution

- If a branch is hard to predict, execute both paths and squash one
- Downsides: always wasting space, actual path competes with squashed path for resource (FUs)



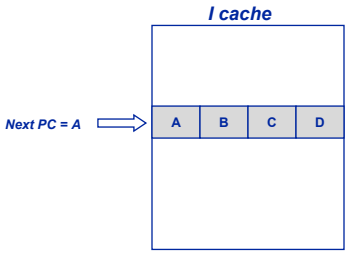
ILP-53

53

High-level Inst Fetch issues

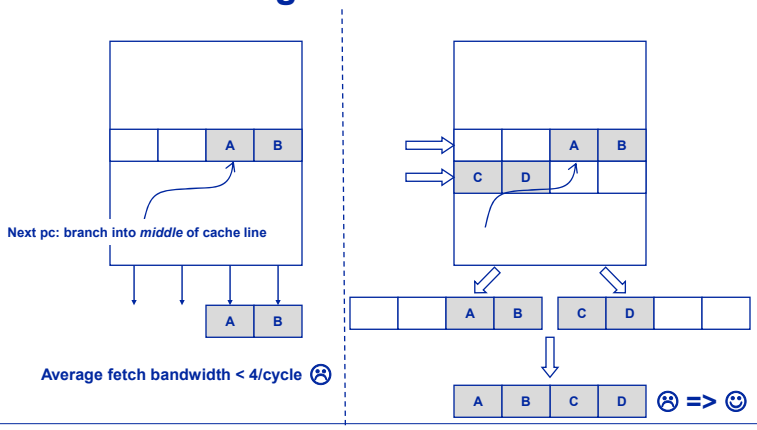
- Challenges:
  1. Need to fetch more than one instruction per cycle to keep superscalar pipeline fed
  2. I$ is single ported– can't read more than one block a cycle
  3. Branches occur every 5 inst. on average, thus need to predict branches for multiple instructions per cycle
- Solutions (not covered here):
  - Collapsing buffer: uses a banked cache and banked branch predictor plus a combining network
  - Trace cache: remembers a path through the code across multiple branches

Instruction fetch and the Icache



- Superscalar fetch needs multiple (e.g., four) instructions per cycle
- In the above, in a traditional cache, fetching A, B, C, D would take four cache accesses
- Better approach: fetch a whole block at a time from the Icache

Problem: unaligned accesses

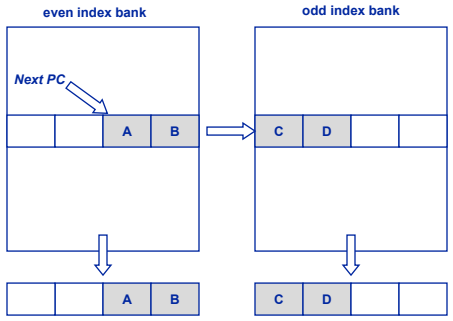


- Solution: dual-port the I$ in order to read out two *consecutive* cache lines
- Merge needed instructions together to get 4 inst./cycle bandwidth
- EXPENSIVE: true dual-porting increases I$ area significantly

ILP-56

56

"Fake" dual porting via dividing the cache into two banks

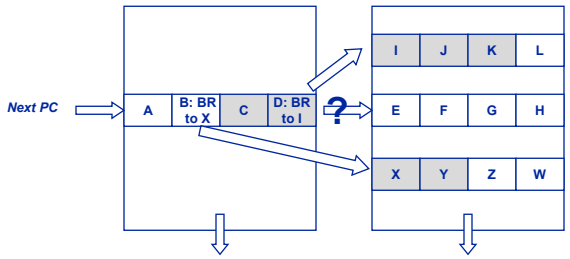


- Scheme is called "Banked Sequential"
- Select to start fetching in bank even or odd by using the least significant bit of the cache index (msb = 0: even, else odd)

ILP-57

57

Next problem: *Predicting* branches!

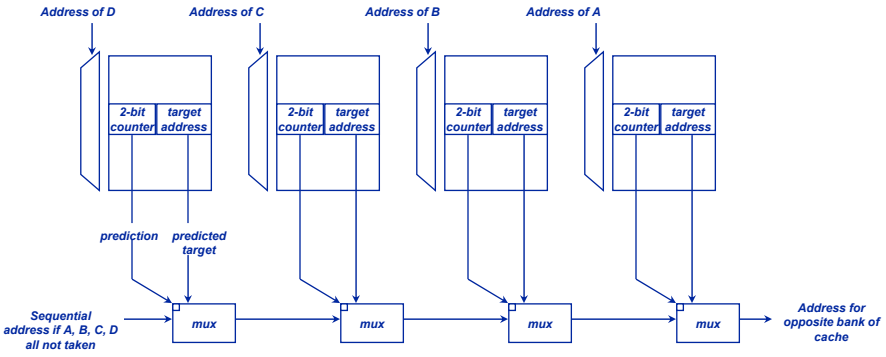


- If either B or D is (predicted) taken, want XYZW or IJKL blocks next, not EFGH block
- Also, if both B is taken and D is taken, want XYZW, not IJKL
  - because B's branch prediction has a "higher priority" than D's branch prediction

ILP-58

58

Interleaved BTB (works with any branch predictor)



- Banks of BTB accessed in parallel
- 1-cycle trip time across "carry chain"
- Overall scheme name: "Banked cache w/BTB"

ILP-59

59

Pipeline Icache Timing and Branch Penalty

Fetched cache blocks *a, b, c, d, e, f, ...*

BTB	*a*	*b*	*c*	*d*	*e*	...	...
Cache data array access		*a/b*	*b/c*	*c/d*	*d/e*	*e/f*	...
Align, tag compare			*a/b*	*b/c*	*c/d*	*d/e*	*e/f*

For simplicity, each block is fetched twice

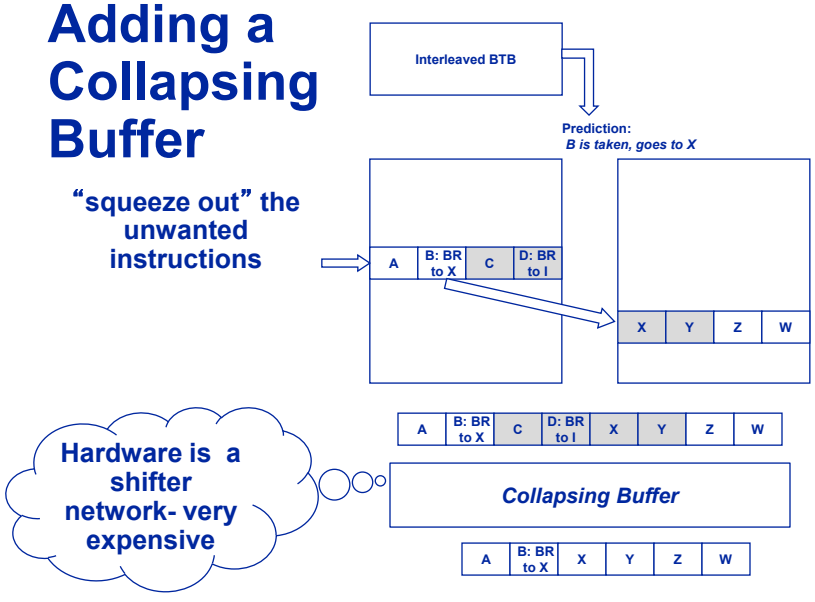
Branch prediction penalty: adds two stages to ifetch, so
2 (=Array+Align) + (stages from decode to when branch direction is resolved)

ILP-60

60

Adding a Collapsing Buffer

"squeeze out" the unwanted instructions



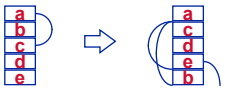
ILP-61

61

Effects of compiler optimizations on superscalar instruction fetch

- Instruction layout enhances Icache performance
  - Use instruction layout from cache miss rate reduction, e.g.,

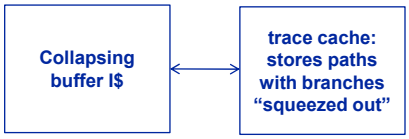
a c d e preferred path (determined by profiling)



- Does instruction layout help superscalar fetch?
- Experimental results:
  - Integer benchmarks from SPEC92
  - 5 profiling inputs, 1 (separate) test input for results
  - Net result: Instruction layout can make banked sequential perform as well as the collapsing buffer

Modification: add a "trace cache"

- Idea:



- Cache the output of the collapsing buffer as "traces"
- Look up (target of previous trace address, branch prediction) in trace $
- If a trace doesn't exist, use collapsing buffer I$ to build new trace